

COMP0008: Computer Architecture & Concurrency – CW3 Solution

Part 1: Scrutinizing MIPS's ISA:

In the lecture, we discussed the difference between `addi` and `addiu`.

- Write a short MIPS assembly program that throws an exception when executing the following instruction:

`addi $t0, $t0, -1`

Solution:

`lui $t0, 0x8000` #set \$t0 to be the smallest sign 32-bit integer: -2^{31} .

`addi $t0, $t0, -1`

- Change the above to `addiu` and check that no exception is thrown. Can you give an example of a case where such an instruction is useful?

Solution:

Consider the following C code:

```
for (unsigned int x = (1<<31) + 5; x < (1<<31) - 5; --x){ // x runs from  $2^{31} + 5$  down to  $2^{31} - 5$   
    // do something  
}
```

*Notice that the **unsigned** representation of 2^{31} is 0x80000000, same as the **2's complement** representation of -2^{31} .*

The compiler's way of letting the CPU know that this is an unsigned integer is to use `addiu` which doesn't throw an exception.

In practice, many compilers may opt not to throw an exception either way. For example, in C, a signed overflow is considered an "undefined behaviour", which means that the compiler is free to ignore this possibility. In fact, in newer MIPS designs the `addi` instruction no longer exists (and its opcode serves a new instruction).

Part 2: Theoretical Questions:

Which of the following claims are correct?

- ❖ It is better to write an assembly program that uses as few registers as possible (even if it means performing more instructions for the same task).

If we are talking about a complete program (as opposed to a function), there is no benefit in minimizing the number of registers we use, and it is better to optimize the number of instructions.

- ❖ Say that programs A and B each execute exactly 1000 instructions to perform a task, but A only uses \$t0 while B modifies many more registers. Then, A will be faster than B due to caching.

This is incorrect: caches are designed to speed up access to slower memories (e.g., main memory). The registers are the fastest memory that the CPU can use and therefore are not cached anywhere.

- ❖ In a function, the callee is responsible for restoring the values of \$a0-\$a3 before the function ends.

No, as shown in the MIPS reference card.

- ❖ If a function does not call another function, it is more efficient to avoid storing \$ra on the stack.

This is correct, as \$ra is not modified unless we call a different function.

- ❖ If a function does not call another function, we should not use the memory for efficiency (as we don't have to modify any of the stored registers).

This is incorrect. There are cases where the computation required in the function needs more variables than the ones we have, and we must store the remaining in memory.

- ❖ Using jal label is essentially the same as doing (1) la \$t0, label and (2) jalr \$t0.

This is incorrect. jal is more efficient as it is encoded as a single instruction. Additionally, it does not modify \$t0.

Part 3: Recursion in MIPS:

In the lecture, we discussed additive (e.g., the Roman) and positional (e.g., decimal) number systems.

In some older additive systems (e.g., [the Egyptian](#)), there was no importance to the order of symbols.

Here, we will consider the Weird0008 number system. It is an additive number system that uses the digits 1-9 (i.e., without zero).

The Weird0008 system requires that digits will be sorted in non-increasing order, and thus allows multiple representations of the same numbers.

Your task is to write a MIPS program that takes as input a number $n > 0$ and output all its Weird0008 representations.

For example, if $n = 5$, the output can be:

5

41

32

311

221

2111

11111

When programming, try to follow the register usage convention (use \$t0-\$t9 for temporary variables, \$s0-\$s1 for saved variables, \$a0-\$a3 for arguments, etc.).

In this CW, I did not explicitly state what are the metrics on which your code will be judged (other than a correct use of the usage convention), since the CW is not marked.

If I'd want you to write a code in a specific way in the exam (e.g., an efficient code or a code that doesn't modify specific registers), I will be super explicit about that.

Below is my solution, which has not been optimized in any way.

In it, we use a recursion function that takes three arguments:

- 1. The sum of the remaining digits (originally, the input number), placed in \$a0.*
- 2. The number of digits in the current prefix (originally, 0), placed in \$a1.*
- 3. The largest digit that's allowed to maintain monotonicity (originally, 9), placed in \$a2.*

In the recursion, we loop through all the options for the next digit (between 1 and $\min(\$a0, \$a2)$).

(But first, we save all the required registers!)

For each possible digit, we:

- 1. write it to $\text{mem}[\$a1]$ (where mem is an auxiliary array),*
- 2. set $\$a0 \leftarrow \$a0$ minus the current digit*
- 3. increment \$a1*
- 4. set $\$a2 \leftarrow \min(\$a2, \text{the current digit})$*
- 5. call the function recursively*

routine to print the numbers on one line.

```
.data
space:      .ascii " "      # space to insert between numbers
newline:    .ascii "\n"
head:       .ascii "\n-----\n"
mem:        .word 0 : 1000   # An auxiliary array
.text

li $8, 15    # The number we wish to represent
la $a0, head # Load the address to print the heading
li $v0, 4    # specify Print String service
syscall      # print heading

move $a0, $8 #The sum of digits should equal $8
li $a1, 0    #The number of digits in the current number
li $a2, 9    #The largest allowed digit
jal recursion # call print routine.
li $v0, 10   # system call for exit
syscall      # we are out of here.

recursion:
    addi $sp, $sp, -16      # make room to store the registers
    sw $ra, 0($sp)         # we're calling a function, must save $ra
    sw $s0, 4($sp)         # by convention, must store $s0 to use it
    sw $s1, 8($sp)         # by convention, must store $s1 to use it
    sw $s2, 12($sp)        # by convention, must store $s2 to use it
    move $s1, $a1          # we need to remember the number of digits so far
    bnez $a0, non_zero     # a pseudo instruction, same as bne $a0, $0
    la $t0, mem            # if we got here, we're done ($a0=0) time to print

print_loop:                # print $a1 digits from mem
    lw $t1, ($t0)          # iterate over mem. $t0 is the pointer, $t1 is the value
    li $v0, 1              # syscall 1 -- print int
    move $a0, $t1          # by convention, the argument goes to $a0
    syscall                # print $t1=mem[$t0/4]
    addi $t0, $t0, 4       # ++t0
    subi $s1, $s1, 1       # --$s1, that tracks how many digits are left
    bgtz $s1, print_loop   # if we haven't printed all digits, go to the next one
    la $a0, newline        # load address of print heading, we're done with one representation.
    li $v0, 4              # specify Print String service
    syscall                # print newline
    j end_recursion        # go restore values and then return to caller
```

non_zero:	# We have (\$a0 > 0), so we're not done
move \$s0, \$a0	# We need to store \$a0 before making rec. calls
move \$s2, \$a2	# set \$s2 = \$a2 (we want \$s2 = min(\$a0, \$a2))
bge \$s0, \$s2, digit_loop	# If (\$s0 >= \$s2) then we are done computing the min
move \$s2, \$s0	# otherwise fix \$s2 to \$s0
 digit_loop:	 # We loop through \$s2, \$s2-1,...,1 for the next digit
sll \$t0, \$s1, 2	# \$t0 = \$s1 * 4 → the byte offset for writing the next digit
la \$t1, mem(\$t0)	# Get \$t1 to point at that memory address
sw \$s2, (\$t1)	# Store the current digit in mem[\$s1]
sub \$a0, \$s0, \$s2	# The remaining digits need to sum up to \$s0-\$s2
move \$a2, \$s2	# For the recursion, we place the max allowed digit in \$a2
addi \$a1, \$s1, 1	# We have now one more digit
jal recursion	# Solve the problem for representing \$a0, with the current prefix
addi \$s2, \$s2, -1	# We're done with all options with digit \$s2, decrement it
bgtz \$s2, digit_loop	# If \$s2>0 we need to continue the loop
 end_recursion:	
lw \$ra, 0(\$sp)	# read registers from stack
lw \$s0, 4(\$sp)	
lw \$s1, 8(\$sp)	
lw \$s2, 12(\$sp)	
addi \$sp, \$sp, 16	# bring back stack pointer
jr \$ra	# return